

HTML5 — Structure & sémantique

Développement web — Classe passerelle

Semaine 2 | **Module B**

Anne Terrier | cfpt·i — École d'informatique

Au programme aujourd'hui

1. Balises et imbrication
2. Structure & sémantique
3. Texte, liens & images

Puis, au milieu

Un exercice : **votre première page web.**

4. Listes & tableaux
5. Valider son HTML

Objectif de la semaine

Écrire une page HTML5 valide et sémantique, et savoir corriger les erreurs signalées par le validateur.

Le polycopié va plus loin

Ces diapositives ne gardent que l'essentiel. Le détail — tableaux de balises, cas particuliers, lexique — est dans *S02_polycop.*

Où en sommes-nous ?

HTML

la structure

semaines 2 et 3

CSS

la présentation

semaines 4 et 5

JavaScript

le comportement

semaines 6 et 7

Aujourd'hui, on ne fait que du HTML

Votre page sera laide. C'est normal, et c'est même voulu : tant qu'il n'y a pas de style, on ne peut juger que la **structure**.

1. Balises et imbrication

Un langage de balisage

Définition

Le **HTML** décrit la **structure** et le **contenu** d'une page. Un simple fichier texte, d'extension `.html`.

Pas un langage de programmation

Ni variables, ni conditions, ni boucles. Tout cela arrivera avec JavaScript, en **semaine 6**.

`<p> Bonjour tout le monde </p>`

↑
Balise ouvrante

↑
Contenu

↑
Balise fermante

Certaines balises n'encadrent rien et s'écrivent **seules** : `<img>`, `<br>`, `<meta>`, `<link>`.

Attributs, espaces et commentaires

```
<a href="https://www.hepia.ch" target="_blank">Site de la hepia</a>

<!-- Un commentaire : invisible dans la page,
      mais lisible par tous via Ctrl+U -->

<p>Ce      texte      s'affichera      normalement.</p>
```

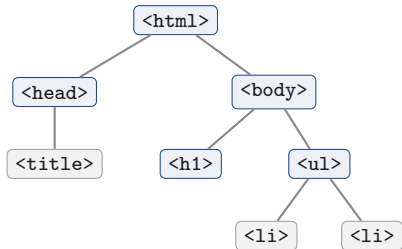
Un attribut

S'écrit `nom="valeur"`, dans la balise **ouvrante** seulement. Jamais dans la fermante.

Le HTML ignore votre mise en page

Espaces multiples et retours à la ligne sont réduits à **un seul espace**. Un paragraphe se crée avec un `<p>`, pas avec Entrée.

L'imbrication : un arbre, une règle



Premier ouvert, dernier fermé

Comme des parenthèses. Les balises ne se chevauchent jamais.

OUI `<p>Un <strong>mot</strong> ici</p>`

NON `<p>Un <strong>mot</p></strong> ici`

Le réflexe qui évite tout

Indenter : un niveau par niveau d'imbrication.
Une balise oubliée devient visible à l'œil nu.

Cet arbre sera parcouru par le navigateur, les moteurs de recherche, et votre JavaScript dès la semaine 7.

2. Structure & sémantique

Le squelette d'une page HTML5

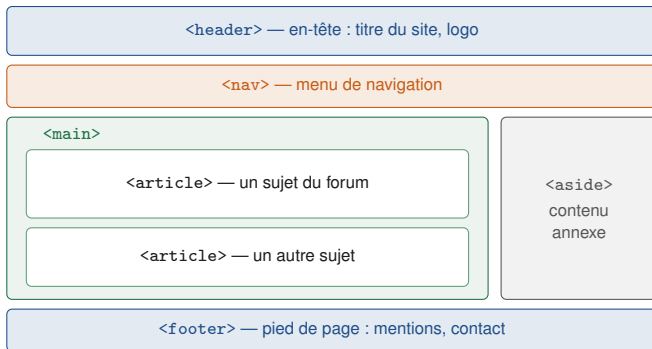
```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>MiniForum</title>
</head>
<body>
  <!-- Tout le contenu visible de la page -->
</body>
</html>
```

<head> et <body> : ne pas confondre

Le <head> contient des informations *sur* la page — encodage, titre d'onglet, feuille de style. Le <body> contient tout le **visible**. Un titre écrit dans le <head> n'apparaîtra jamais dans la page.

Squelette à connaître **par cœur** — demandé au contrôle écrit de la semaine 12.

Décrire le rôle de chaque zone



Une balise sémantique dit ce que le contenu **est**, pas comment il s'affiche.

Pourquoi s'en soucier — et quel choix faire

Trois publics en tirent parti

- les **humains** qui relisent le code
- les **moteurs de recherche**
- les **lecteurs d'écran**

Déroutant au début

Remplacer un `<div>` par un `<main>` **ne change rien à l'affichage**. Le bénéfice est invisible — et bien réel.

`<section>` ou `<article>` ?

Le test : ce contenu garderait-il du sens **sorti de la page** ?

Oui → `<article>` : un message de forum, un billet.

Non → `<section>` : « Sujets récents » n'existe que dans ce forum.

Et le `<div>` ?

Il reste utile quand il n'y a **aucun sens** à exprimer, seulement une mise en forme.

3. Texte, liens & images

```
<h1>MiniForum</h1>
<h2>Sujets récents</h2>
<p>Un paragraphe avec du <strong>contenu important</strong>.</p>
```

La hiérarchie des titres

Les titres font le **plan** du document, pas la taille du texte.

Un seul <h1> par page, et **jamais de saut** de niveau.

 plutôt que

L'un décrit un *sens*, l'autre une *apparence*.
Rendu identique — mais un lecteur d'écran change d'intonation sur et ignore . Idem face à <i>.

Au passage : <p>, <h1>, sont des éléments **en bloc** (toute la largeur) ;
<a>, , sont **en ligne** (dans le fil du texte). On y reviendra en semaine 4.

Liens et identifiants

```
<a href="https://www.hepia.ch">Site de la hepia</a>      <!-- externe -->
<a href="sujet.html">Voir le sujet</a>                  <!-- autre page -->
<a href="#sujets">Aller aux sujets</a>                  <!-- meme page -->

<section id="sujets"> ... </section>
```

Un id est unique

Jamais deux fois le même dans une page
— le validateur signale les doublons.

Il ressortira au `<label>` (sem. 3), au **CSS** (sem. 4), à **JavaScript** (sem. 7).

Un bon intitulé de lien

Il doit décrire sa destination **hors**

contexte : les lecteurs d'écran listent les liens d'une page.

OUI Lire le sujet **NON** cliquez ici

Chemins et images

```
miniforum/  
|-- index.html  
|-- sujet.html  
\-- images/  
    |-- logo.png
```

```

```

Chemins : la source d'erreur n° 1

sujet.html même dossier | images/logo.png
sous-dossier | ../style.css dossier parent

La casse compte sur un serveur Linux : ça marche chez vous, ça casse en ligne. **Ni espaces ni accents** dans les noms.

L'attribut alt n'est pas optionnel

Lu par les lecteurs d'écran, affiché si l'image ne charge pas. Son absence est une **erreur** pour le validateur.

OUI alt="Logo du MiniForum"

NON alt="image"

À vous de jouer !

Exercice intermédiaire — votre première page web

Exercice — votre première page web

Consignes — fichier `ma-page.html`

1. Le squelette écrit **à la main**
2. Un `<header>` avec un `<h1>` : votre prénom
3. Un `<main>` avec une `<section>` : un `<h2>`, un paragraphe, un mot en `<strong>`
4. Une image, avec un `alt` correct
5. Un lien à l'intitulé explicite
6. Un `<footer>` avec la date du jour

Ce que je vérifie en passant

- Le code est **indenté**
- **Un seul** `<h1>`
- L'`alt` décrit quelque chose
- Le chemin de l'image est **relatif**
- Aucune balise laissée ouverte

En avance ?

Ajoutez une liste `<ul>` et une `<figure>` avec sa `<figcaption>`.

20 minutes, sans le raccourci !+Tab — puis on valide ensemble sur validator.w3.org.

Exercice — une correction possible

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Alice - présentation</title>
</head>
<body>
  <header><h1>Alice</h1></header>
  <main>
    <section>
      <h2>Qui suis-je ?</h2>
      <p>Étudiante en classe passerelle, je découvre le
        <strong>développement web</strong> cette année.</p>
      
      <p><a href="https://developer.mozilla.org">La documentation MDN</a></p>
    </section>
  </main>
  <footer><p>Page créée le 15 septembre 2026</p></footer>
</body>
</html>
```

4. Listes & tableaux

Liste à puces — ordre indifférent

```
<ul>  
  <li>HTML </li>  
  <li>CSS </li>  
  <li>JavaScript </li>  
</ul>
```

Liste numérotée — l'ordre compte

```
<ol>  
  <li>Ouvrir le terminal </li>  
  <li>Taper la commande </li>  
  <li>Valider </li>  
</ol>
```

Un `<li>` vit toujours dans un `<ul>` ou un `<ol>`

Rien d'autre qu'un `<li>` ne peut être enfant direct d'un `<ul>` : pas de `<p>`, pas de `<div>`.
Ce contenu se place à l'intérieur du `<li>`.

Un menu de navigation est une liste de liens

```
<nav>
  <ul>
    <li><a href="index.html">Accueil</a></li>
    <li><a href="sujet.html">Sujets</a></li>
  </ul>
</nav>
```

Étrange ? Pas tant que ça

Cela paraît lourd tant qu'on pense en termes d'**apparence**. Mais un menu *est* bien une liste de liens — et le CSS se chargera, en semaine 4, de l'afficher horizontalement.

C'est exactement l'esprit du HTML : **décrire le sens**, laisser le CSS décider de la forme.

```
<table>
  <caption>Derniers sujets publiés</caption>
  <thead>
    <tr><th>Sujet</th><th>Auteur</th></tr>
  </thead>
  <tbody>
    <tr><td>Bienvenue</td><td>Alice</td></tr>
    <tr><td>Installer Node.js</td><td>Bob</td></tr>
  </tbody>
</table>
```

Un tableau sert à des **données**, jamais à mettre en page

Horaire, relevé de notes, comparatif : oui. Positionner des éléments dans la page : non — c'est le rôle du CSS. Une page mise en page avec des tableaux est illisible pour un lecteur d'écran.

5. Valider son HTML

Pourquoi valider

Le navigateur est trop gentil

Il affiche une page même si des balises sont mal fermées : il **devine** ce que vous avez voulu écrire.

Et c'est un piège

Chaque navigateur devine à sa manière. Le jour où l'affichage se dérègle, l'erreur est déjà enfouie sous cent lignes de code — et elle ne se manifeste que chez certains utilisateurs.

Le validateur du W3C — `validator.w3.org`

Trois modes : par **adresse**, par **fichier** téléversé, ou par **copier-coller** — c'est celui qu'on utilise en classe.

Les messages fréquents — et la bonne méthode

Message	Ce qu'il faut corriger
Unclosed element X	La balise X n'est jamais fermée
Stray end tag X	Une fermante sans ouvrante
Duplicate ID "xyz"	Deux éléments portent le même id
An img element must have an alt attribute	Une image sans alt
Element X not allowed as child of element Y	Mauvaise imbrication

Corriger de haut en bas

Une seule balise oubliée produit dix messages en cascade : corrigez **la première**, relancez, le compteur chute d'un coup. Les **Errors** doivent toutes disparaître ; les **Warnings** sont des maladresses.

Prévenir plutôt que corriger

Trois habitudes qui suppriment la majorité des erreurs

- **Indenter** systématiquement — Maj+Alt+F dans VS Code, avec Prettier
- Écrire la balise **fermante immédiatement** après l'ouvrante, puis remplir le contenu
- Se fier au **soulignement rouge** de VS Code, qui repère déjà une partie des erreurs

Le réflexe documentation

MDN Web Docs reste la première source : cherchez `mdn` suivi du nom de la balise. La page donne la description, la syntaxe, et surtout des **exemples modifiables**.

Ce qu'on a vu

- Le HTML, langage de balisage
- L'imbrication, et l'arbre du document
- Le squelette : `<head>` / `<body>`
- Les balises sémantiques
- Titres, texte, liens, images, chemins
- Listes et tableaux
- La validation W3C

À faire pour la semaine 3

- Les **deux pages** du MiniForum (fiche TP)
- Chacune valide **sans erreur**, poussée sur GitHub
- Le quiz d'auto-évaluation sur Moodle

La semaine prochaine

Les **formulaires** et l'**accessibilité** — encore du HTML, et la dernière semaine avant le CSS.

Des questions ?

Prochaine étape : la fiche TP — structure HTML du MiniForum